Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)
EXPERIMENT NO. 5

AIM:
Establish a database layer using Prisma ORM with PostgreSQL/MySQL to wire relationship logic and handle full transactional operations.

SOFTWARE REQUIRED:
- **Operating System:** Linux / macOS / Windows 10/11
- **Container Engine:** Docker Engine & Docker Compose (or Docker Desktop)
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code

THEORY:
- **Containerized Database Infrastructure (Docker Compose):** Docker enables isolated, reproducible development environments. Using Docker Compose, an isolated PostgreSQL database engine runs with persistent named storage volumes (postgres_data), ensuring consistent local infrastructure without requiring manual database installations on host operating systems.
- **Prisma 7 Architecture & Programmatic Configuration (**prisma.config.ts**):** In Prisma 7, datasource URLs are decoupled from declarative schema.prisma files and managed through a TypeScript configuration file (prisma.config.ts). This allows environment parameters to be resolved dynamically and type-safely at runtime and build time.
- **Driver Adapters (**@prisma/adapter-pg**):** Prisma 7 replaces legacy internal binary query engines with JavaScript/TypeScript database driver adapters. Using @prisma/adapter-pg alongside the native Node.js pg connection pool (pg.Pool), queries execute through standardized connection pools optimized for serverless, edge, and containerized runtime environments.
- **Relational Schema Modeling (1:N):** Entities are modeled declaratively. A User (Author) connects to multiple Post records using @relation(fields: [authorId], references: [id], onDelete: Cascade). Cascading rules enforce referential integrity across related tables when parent records are modified or removed.
- **ACID Transactions (**prisma.$transaction**):** Transactions guarantee that a sequence of database operations either completely succeed or completely roll back in the event of an error, preserving Atomicity, Consistency, Isolation, and Durability. Interactive transactions (prisma.$transaction(async (tx) => { ... })) ensure dependent multi-table writes execute inside an isolated lock context.
- **Global Singleton Connection Pattern:** Next.js Hot Module Replacement (HMR) re-evaluates server modules on every file change during development. Attaching PrismaClient to the Node.js globalThis object prevents the creation of redundant client instances and prevents exhausting the PostgreSQL connection pool.

STEPS / ALGORITHM:

Step 1: Configure PostgreSQL Container (docker-compose.yml**)**

Create a docker-compose.yml file in the root directory to provision an isolated PostgreSQL instance:

```
services:
  postgres-db:
    image: postgres:16-alpine
    container_name: nextjs_postgres_db
    restart: always
    environment:
      POSTGRES_USER: devuser
      POSTGRES_PASSWORD: devpassword
      POSTGRES_DB: nextjs_blog_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Start the container in detached mode:

```
docker compose up -d
```

Step 2: Install Prisma 7, PostgreSQL Driver Adapter, and Dependencies

Install Prisma 7, the PostgreSQL adapter, and native driver bindings:

```
npm install @prisma/client @prisma/adapter-pg pg
npm install prisma @types/pg --save-dev
```

Configure your connection string inside .env:

```
DATABASE_URL="postgresql://devuser:devpassword@localhost:5432
/nextjs_blog_db?schema=public"
```

Step 3: Define Prisma 7 Configuration (prisma.config.ts**)**

Create prisma.config.ts in the project root to configure the datasource programmatic entry point:

```
import { defineConfig } from 'prisma';

export default defineConfig({
  datasource: {
    url: process.env.DATABASE_URL,
  },
});
```

Step 4: Define Relational Data Models (prisma/schema.prisma**)**

In Prisma 7, the datasource db block specifies only the provider. Define the 1:N relation between User and Post:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

model User {
  id        String   @id @default(cuid())
  name      String
  email     String   @unique
  posts     Post[]
  createdAt DateTime @default(now())
}

model Post {
  id        String   @id @default(cuid())
  title     String
  content   String
  published Boolean  @default(true)
  authorId  String
  author    User     @relation(fields: [authorId],
references: [id], onDelete: Cascade)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Run schema migrations and generate client types:

```
npx prisma migrate dev --name init_prisma7_schema
npx prisma generate
```

Step 5: Instantiate Driver Adapter & Singleton Client (app/lib/prisma.ts**)**
Initialize the PostgreSQL connection pool using pg.Pool, bind it to PrismaPg, and attach the client instance to globalThis:

```
import { PrismaClient } from '@prisma/client';
import { PrismaPg } from '@prisma/adapter-pg';
import { Pool } from 'pg';

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

const adapter = new PrismaPg(pool);

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    adapter,
    log: ['query', 'error', 'warn'],
  });

if (process.env.NODE_ENV !== 'production')
globalForPrisma.prisma = prisma;
```

Step 6: Implement Transactional Server Actions (app/actions/blogDbActions.ts**)**
Create an interactive ACID transaction that atomically creates both the Author (User) and initial Post:

```
'use server';

import { revalidatePath } from 'next/cache';
import { prisma } from '@/app/lib/prisma';

export async function createAuthorWithPostAction(formData:
FormData) {
```

```
  const name = formData.get('name') as string;
  const email = formData.get('email') as string;
  const title = formData.get('title') as string;
  const content = formData.get('content') as string;

  if (!name || !email || !title || !content) {
    throw new Error('All fields are required.');
  }

  // Interactive ACID Transaction
  await prisma.$transaction(async (tx) => {
    // Step A: Upsert Author
    const user = await tx.user.upsert({
      where: { email },
      update: { name },
      create: { name, email },
    });

    // Step B: Create Post associated with the resolved
author ID
    await tx.post.create({
      data: {
        title,
        content,
        authorId: user.id,
      },
    });
  });

  revalidatePath('/blog-db');
}

export async function deletePostAction(postId: string) {
  await prisma.post.delete({
    where: { id: postId },
  });

  revalidatePath('/blog-db');
}
```

Step 7: Construct Relational UI View (app/blog-db/page.tsx**)**

Create an asynchronous Server Component to fetch relational records and render the form:

```
import { prisma } from '@/app/lib/prisma';
import { createAuthorWithPostAction, deletePostAction } from
'@/app/actions/blogDbActions';

export default async function BlogDbPage() {
  const posts = await prisma.post.findMany({
    include: {
      author: true,
    },
    orderBy: {
      createdAt: 'desc',
    },
  });

  return (
    <div style={{ maxWidth: '800px', margin: '0 auto',
padding: '20px', fontFamily: 'sans-serif' }}>
      <h1>Experiment 5: Prisma 7 & Docker PostgreSQL</h1>
      <p style={{ color: '#666' }}>Demonstrating Prisma 7
Programmatic Config, Relational Joins, and ACID
Transactions.</p>

      {/* Transactional Form */}
      <div style={{ background: '#f4f4f5', padding: '20px',
borderRadius: '8px', marginBottom: '30px' }}>
        <h3 style={{ margin: '0 0 10px 0' }}>Create Author &
Post (Atomic Transaction)</h3>
        <form action={createAuthorWithPostAction} style={{
display: 'grid', gap: '10px' }}>
          <div style={{ display: 'flex', gap: '10px' }}>
            <input type="text" name="name"
placeholder="Author Name" required style={{ flex: 1, padding:
'8px', borderRadius: '4px', border: '1px solid #ccc' }} />
            <input type="email" name="email"
placeholder="Author Email" required style={{ flex: 1,
padding: '8px', borderRadius: '4px', border: '1px solid #ccc'
}} />
          </div>
```

```
          <input type="text" name="title" placeholder="Post
Title" required style={{ padding: '8px', borderRadius: '4px',
border: '1px solid #ccc' }} />
          <textarea name="content" placeholder="Post Content"
required rows={3} style={{ padding: '8px', borderRadius:
'4px', border: '1px solid #ccc' }} />
          <button type="submit" style={{ padding: '10px',
background: '#09090b', color: '#fff', border: 'none',
borderRadius: '4px', cursor: 'pointer' }}>
            Submit Transaction
          </button>
        </form>
      </div>

      {/* Relational Data Rendering */}
      <h3 style={{ margin: '0 0 15px 0' }}>Database Posts
(Relational Joined View)</h3>
      <div style={{ display: 'flex', flexDirection: 'column',
gap: '15px' }}>
        {posts.length === 0 ? (
          <p style={{ color: '#777' }}>No records found.
Submit the form above to trigger a transaction.</p>
        ) : (
          posts.map((post) => (
            <div key={post.id} style={{ border: '1px solid
#e4e4e7', padding: '15px', borderRadius: '6px', display:
'flex', justifyContent: 'space-between', alignItems: 'center'
}}>
              <div>
                <h4 style={{ margin: '0 0 5px 0'
}}>{post.title}</h4>
                <p style={{ margin: '0 0 8px 0', color:
'#4b5563' }}>{post.content}</p>
                <span style={{ fontSize: '12px', background:
'#e0e7ff', color: '#3730a3', padding: '3px 8px',
borderRadius: '12px' }}>
                  Author: {post.author.name}
({post.author.email})
                </span>
              </div>
```

```
                <form action={deletePostAction.bind(null,
post.id)}>
                  <button type="submit" style={{ background:
'#ef4444', color: '#fff', border: 'none', padding: '6px
12px', borderRadius: '4px', cursor: 'pointer' }}>
                    Delete
                  </button>
                </form>
              </div>
            ))
          )}
        </div>
      </div>
  );
}
```

Step 8: Validate Database & Visual Inspection

1. Run Prisma Studio in a dedicated terminal:

```
npx prisma studio
```

2. Start the development server:

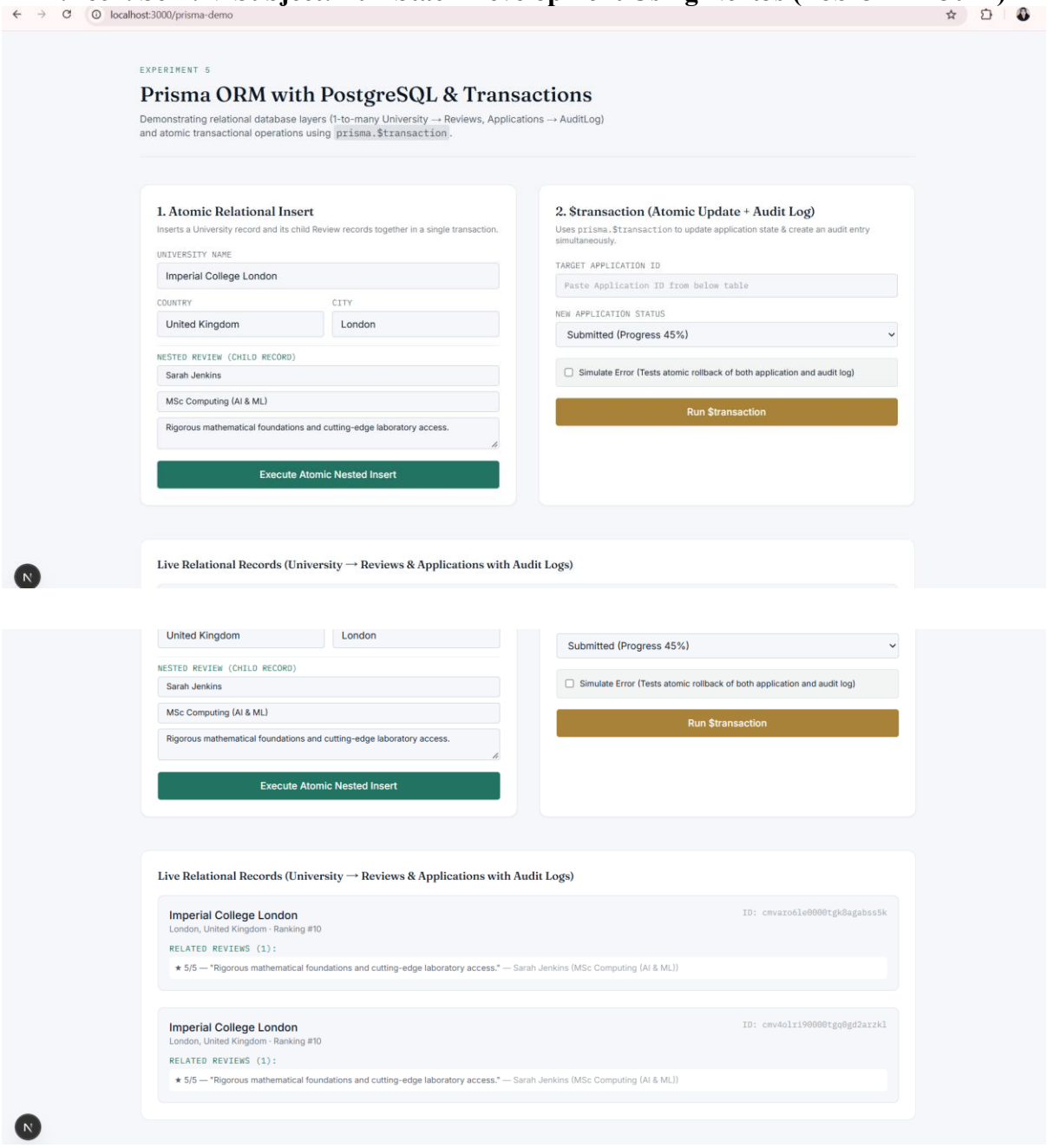
```
npm run dev
```

3. Navigate to http://localhost:3000/blog-db, submit a new record, and verify database writes in the web viewport and in Prisma Studio (http://localhost:5555).

OBSERVATION:

Observe how Prisma 7 integrates with the containerized PostgreSQL database using the @prisma/adapter-pg driver adapter.

When executing mutations, prisma.$transaction() initiates an atomic SQL transaction block (BEGIN -> User.upsert -> Post.create -> COMMIT). If an error occurs during either step, the database state rolls back completely (ROLLBACK), preventing orphan records and ensuring referential integrity across the relation.



CONCLUSION:

A containerized PostgreSQL database layer was successfully established using Docker Compose and integrated with Prisma 7 in Next.js. The experiment verified programmatic configuration (prisma.config.ts), driver adapter integration (@prisma/adapter-pg), 1:N relational modeling, singleton client pooling, and atomic data mutations using interactive ACID transactions (prisma.$transaction).

Department of Artificial Intelligence and Machine Learning

B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)

POST LAB QUESTION:

Q: Why does Prisma 7 require a driver adapter like @prisma/adapter-pg **instead of using the built-in query engine for direct TCP connections?**

A:

Prisma 7 decouples the ORM client from proprietary Rust-based internal network engines in favor of standard, modular JavaScript/TypeScript database driver adapters.

Connection Pool Re-use: It allows Prisma to leverage battle-tested connection pool implementations (like pg.Pool), reducing memory overhead and offering direct control over socket timeouts and max connection limits.

Serverless & Edge Compatibility: Driver adapters enable Prisma to execute across standard Node.js, Edge runtimes (Cloudflare Workers, Vercel Edge), and WebAssembly environments using unified driver interfaces without requiring OS-specific binaries.